

Curso Patrones y Diseño de Software - ITM

Patrones estructurales

Integrantes:

Carlos Hernando Franco Idarraga

Iván Darío Valencia Correa

Edison Estival Restrepo Ospina

Repositorio Front -> <https://github.com/EdiRestrepo/risk-event-notification-fronted.git>

Repositorio Back -> <https://github.com/EdiRestrepo/risk-event-notification-backend.git>

Objetivo: Demostrar la correcta aplicación de patrones estructurales e implementación de estos en una arquitectura moderna.

De acuerdo con el reto seleccionado y el proyecto desarrollado en la primera entrega:

Parte A. Justificación de patrones estructurales (20%):

1. Patrón: Adapter

Funcionalidad: Recibir una alerta con una estructura externa y no controlada, y transformarla (adaptarla) en una estructura propia que pueda tener un mayor control por parte de la aplicación.

Justificación: Desde el servicio externo del SIATA que genera las alertas, generan alertas con una estructura compleja y con información que la API no necesita, por lo que es necesario que esa información se pase por un adaptador para poder convertirlo en un objeto que la api reconozca y pueda manejar.

2. Patrón: Decorator

Funcionalidad: Enriquecimiento de mensajes de alerta

Justificación: Los mensajes de las alertas pueden variar según el nivel de riesgo, ubicación, recomendaciones, prioridad, fecha, mensaje claro de la alerta. Es por ello, que la implementación de este patrón nos ayuda o nos permite agregar información adicional a los mensajes de las alertas de forma flexible y combinable. El decorador “envuelve (Wrapper) las alertas”. Es decir, se asignan responsabilidades adicionales a una alerta de forma dinámica extendiendo la funcionalidad de los mensajes de alerta.

3. Patrón: Facade

Funcionalidad: Orquestación - Centralización de los servicios

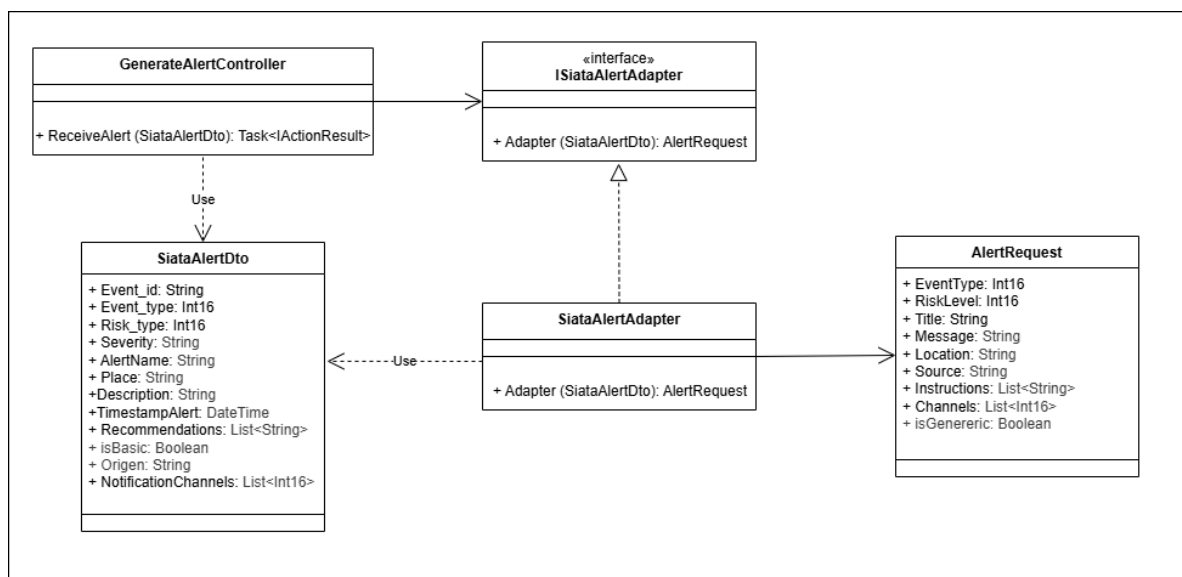
Justificación: Actualmente el cliente en nuestro caso el componente del Dashboard esta haciendo el consumo de diferentes servicios tales como: *NotificationService*, *AlertMessageBuilderService*, *UserPreferencesService*, *Router*.

Con este patron lo que hacemos es centralizar dichos servicios en un solo servicio que va a ser nuestra fachada *AlertCenterFacadeService* de tal forma que el cliente solo haga consumo desde un solo servicio unificado y con la información simplificada con esto se logra **desacoplamiento y simplicidad** ya que el Dashboard depende “solo” de la fachada *AlertCenterFacadeService*.

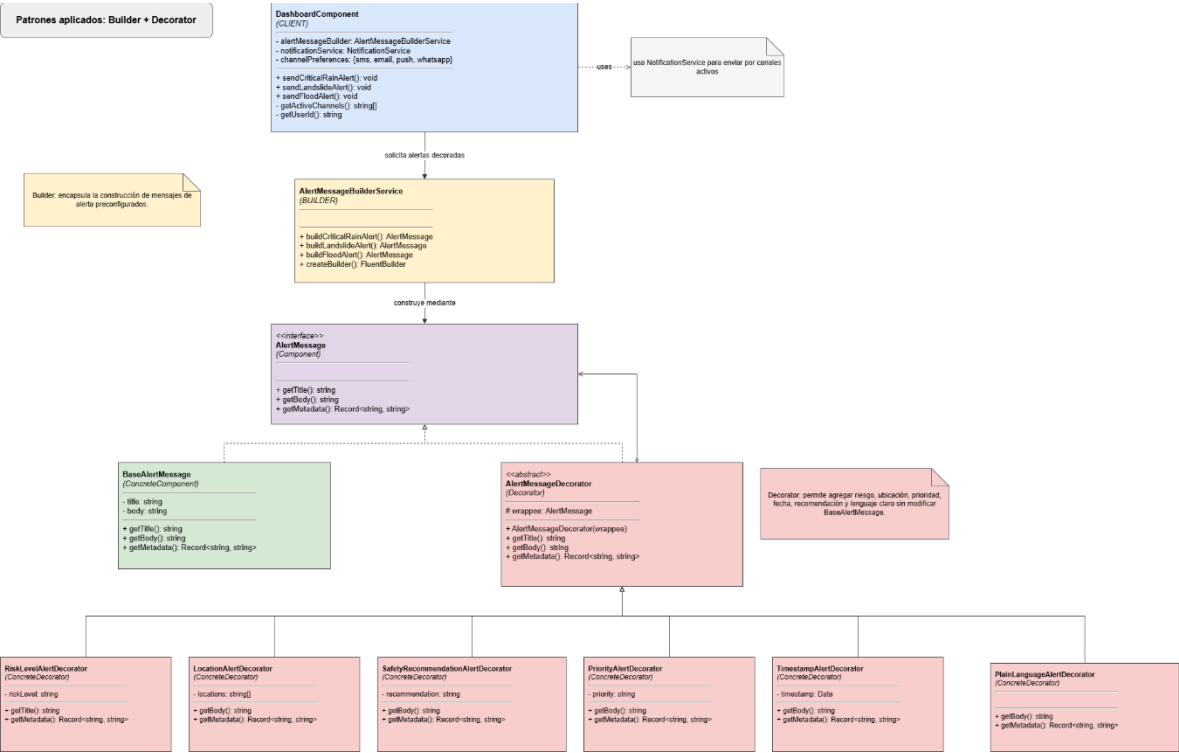
La desventaja es que se puede caer en un antipatrón de una clase God

Parte B. Diagrama UML de Clases (20%)

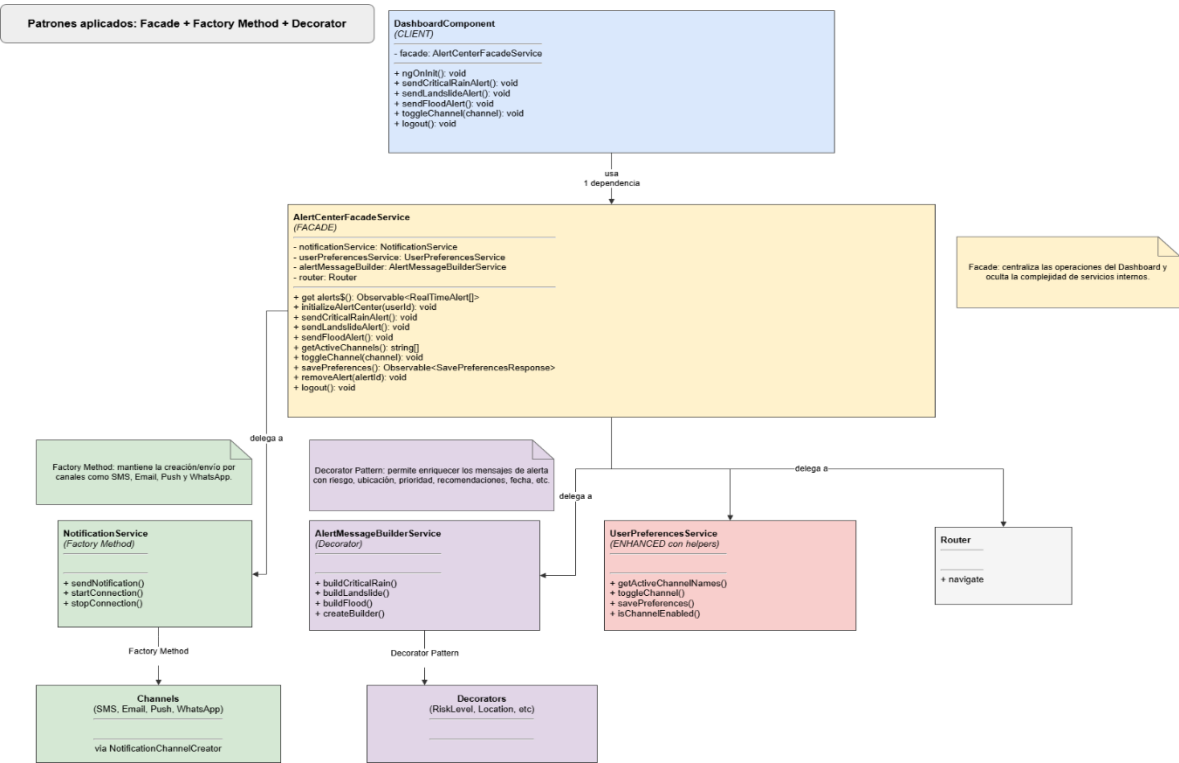
Adapter



Decorator



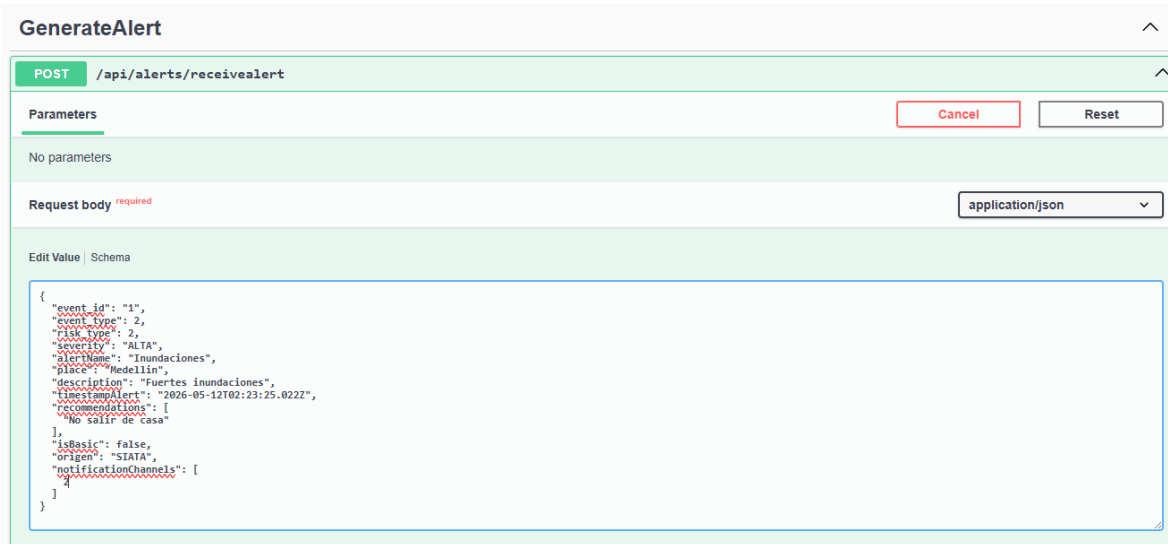
Facade



Parte C. Implementación de los patrones diseño (60%).

Adapter:

Se recibe la alerta del “SIATA”



The screenshot shows a REST client interface for a POST request to `/api/alerts/receivealert`. The request body is a JSON object representing an alert. The interface includes a 'Parameters' section (empty), a 'Request body' section with a dropdown set to 'application/json', and an 'Edit Value' section showing the JSON payload.

```
{
  "event_id": "1",
  "event_type": 2,
  "risk_type": 2,
  "severity": "ALTA",
  "alertName": "Inundaciones",
  "place": "Medellin",
  "description": "Fuertes inundaciones",
  "timestampAlert": "2026-05-12T02:23:25.022Z",
  "recommendations": [
    "No salir de casa"
  ],
  "isBasic": false,
  "origen": "SIATA",
  "notificationChannels": [
    2
  ]
}
```

Objeto Original llegado en la alerta:

```
{
  "event_id": "1",
  "event_type": 2,
  "risk_type": 2,
  "severity": "ALTA",
  "alertName": "Inundaciones",
  "place": "Medellin",
  "description": "Fuertes inundaciones",
  "timestampAlert": "2026-05-12T02:37:22.545Z",
  "recommendations": [
    "No salir de casa"
  ],
  "isBasic": false,
  "origen": "SIATA",
  "notificationChannels": [
    2
  ]
}
```

Se observa como el api la recibe en su controlador

```

16 public GenerateAlertController(IAlerstFacade alertsFacade, ISiataAlertAdapter siataAlertAdapter)
17 {
18     this.alertsFacade = alertsFacade;
19     this.siataAlertAdapter = siataAlertAdapter;
20 }
21
22 [HttpPost("receivealert")]
23 public async Task<ActionResult> ReceiveAlert(SiataAlertDto siataAlertDto)
24 {
25     ResultObject resultObject = new ResultObject
26     {
27         Success = false,
28         Message = String.Empty,
29         Token = Guid.NewGuid()
30     };
31
32     AlertRequest request = this.siataAlertAdapter.Adapter(siataAlertDto);
33
34     Boolean result = await this.alertsFacade.GenerarateAlertAsync(request);
35     if (result == true)
36     {
37         resultObject.Success = true;
38         resultObject.Message = "Alerta Generada Correctametine";
39         return Ok(resultObject);
40     }
41     resultObject.Message = "No se pudo generar la alerta";
42     return BadRequest(resultObject);
43 }

```

siataAlertDto: {1,2,2,ALTA,Inundaciones,Medellin,Fuertes inundaciones,12/05/2026 2:37:22 a.m.,false}

AlertName	View	"Inundaciones"
Description	View	"Fuertes inundaciones"
Event_id	View	"1"
Event_type	View	2
NotificationChannels	View	Count = 1
Origen	View	"SIATA"
Place	View	"Medellin"
Recommendations	View	Count = 1
Risk_type	View	2
Severity	View	"ALTA"
TimestampAlert		{12/05/2026 2:37:22 a.m.}
isBasic		false

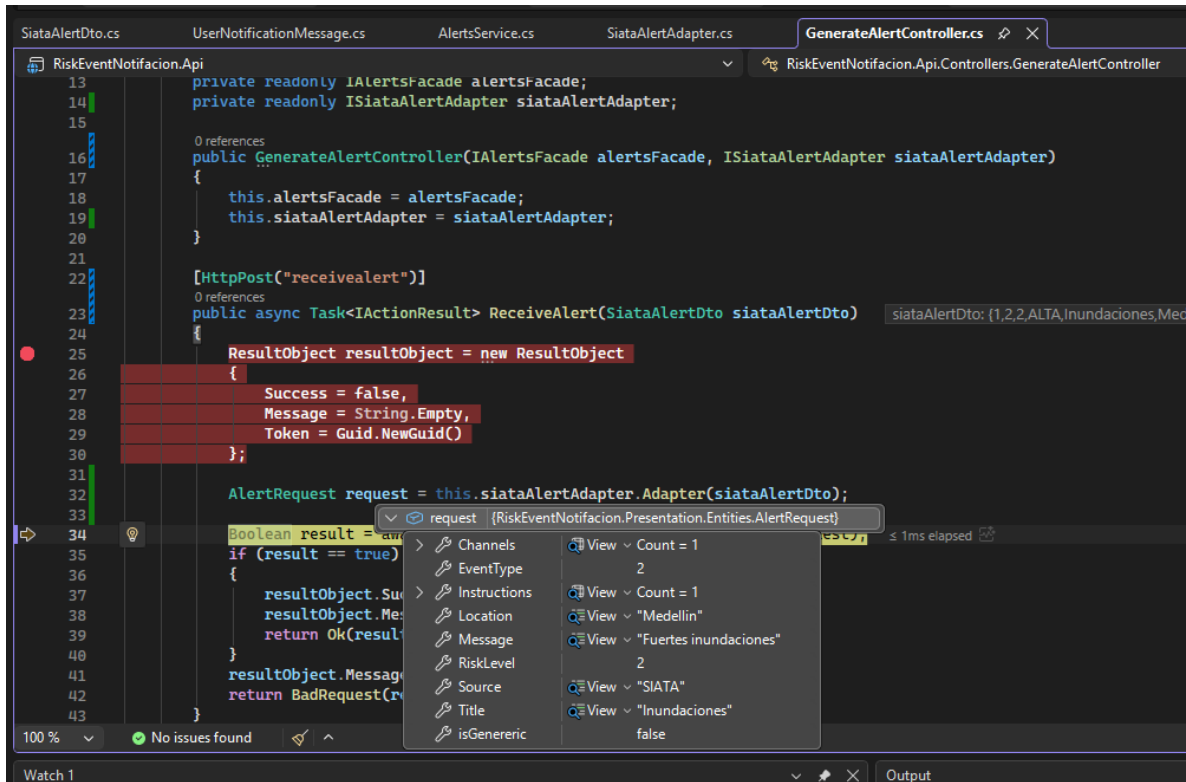
Se pasa por el adaptador para asignar exclusivamente la información que se necesita.

```

1 using RiskEventNotifacion.Presentation.DTOs;
2 using RiskEventNotifacion.Presentation.Entities;
3 using RiskEventNotifacion.Presentation.Interfaces;
4
5 namespace RiskEventNotifacion.Presentation.Services
6 {
7     1 reference
8     public class SiataAlertAdapter : ISiataAlertAdapter
9     {
10         2 references
11         public AlertRequest Adapter(SiataAlertDto siataAlertDto)
12         {
13             return new AlertRequest
14             {
15                 EventType = siataAlertDto.Event_type,
16                 RiskLevel = siataAlertDto.Risk_type,
17                 Title = siataAlertDto.AlertName,
18                 Message = siataAlertDto.Description,
19                 Location = siataAlertDto.Place,
20                 Source = siataAlertDto.Origen,
21                 Instructions = siataAlertDto.Recommendations,
22                 Channels = siataAlertDto.NotificationChannels,
23                 isGenereric = siataAlertDto.isBasic
24             };
25         }
26     }
27 }

```

Se observa el objeto final ya adaptado a lo que necesita la aplicación.



Decorator y Facade

